

Unit-Tests in Notebooks

Unit-Tests prüfen einzelne, in sich geschlossene Codebausteine (meist Funktionen) automatisch auf korrektes Verhalten. Wer früh und regelmäßig testet, findet Fehler schneller und deckt falsche Annahmen im eigenen Code frühzeitig auf. Diese Seite konzentriert sich auf einfache Funktionstests, nicht auf fortgeschrittene Konzepte wie Mocks oder Integrationstests.

Wo werden Funktionen und Tests abgelegt?

Sprache	Empfehlung
Python / R	Funktionen außerhalb von Notebooks in eigenen Dateien speichern – so lassen sie sich wiederverwenden und mit externen Test-Frameworks prüfen
Scala	Funktionen in einem Notebook, Tests in einem separaten Notebook (externe Dateien werden für Scala nicht unterstützt)
SQL	Funktionen als SQL User-Defined Functions (UDFs) in einem Schema ablegen, aus SQL-Notebooks heraus aufrufen und testen

Funktionen und Tests im selben Notebook zu belassen ist zwar einfacher zu pflegen, schränkt aber Wiederverwendbarkeit und externe Testbarkeit ein.

Wie sollten testbare Funktionen aussehen?

- Ein Funktionsergebnis sollte ein einziges, vorhersagbares Ergebnis eines konsistenten Datentyps liefern.
- Existenzprüfungen sollten Boolesche Werte (wahr/falsch) zurückgeben statt bedingter Rückgabewerte.
- Zählfunktionen sollten stets denselben numerischen Typ liefern statt gemischte Rückgabetypen.

Beispiele aus der Dokumentation: eine Funktion, die prüft, ob eine Tabelle existiert; eine, die prüft, ob eine Spalte existiert; eine, die Zeilen mit einem bestimmten Wert zählt.

Python-Beispiel

Funktionen liegen in `myfunctions.py` (hier muss die Spark-Session explizit erzeugt werden, da Notebooks dies automatisch übernehmen, eigenständige Dateien aber nicht). Die Testdatei `test_myfunctions.py` nutzt **pytest**, das automatisch Dateien mit dem Präfix `test_` und Funktionen mit dem Präfix `test_` erkennt.

Wichtige Best Practice: Unit-Tests sollten niemals gegen Produktionsdaten laufen. Stattdessen werden künstliche Testdaten verwendet, die dieselbe Struktur wie die echten Produktionsdaten nachbilden.

```
%pip install pytest
import pytest
retcode = pytest.main(["-v", "-p", "no:cacheprovider"])
```

R-Beispiel

Funktionen liegen in `myfunctions.r` und werden per `source()` in R-Notebooks eingebunden. Das Framework **testthat** erkennt automatisch Dateien, die mit "test" beginnen, und nutzt `test_that()` mit Prüfungen wie `expect_true()`.

Scala-Beispiel

Funktionen liegen in einem eigenen Notebook und werden per `%run` in das Test-Notebook eingebunden. Das Framework **ScalaTest** (Stil `FunSuite`) wird zur Testdurchführung genutzt; auch hier kommen synthetische DataFrames mit passenden Schemata zum Einsatz.

SQL-Beispiel

SQL-UDFs werden mit `CREATE OR REPLACE FUNCTION` in einem bestimmten Katalog/Schema erstellt. Die Hilfsfunktionen `table_exists` und `column_exists` funktionieren nur mit Unity Catalog und befinden sich aktuell in der Public Preview (Vorschauphase). Tests nutzen `SELECT`-Anweisungen mit `CASE` / `IF`-Logik gegen Views (nicht Produktionstabellen).

Test-Frameworks im Überblick

Sprache	Framework	Ausführung
Python	pytest	<code>pytest.main()</code> mit Ausführlichkeits- und Cache-Optionen
R	testthat	<code>test_dir()</code> mit TAP-Reporting (Test Anything Protocol)
Scala	ScalaTest	Testklasse instanziiieren und ausführen, Ergebnisse inline anzeigen
SQL	Eigene SELECT-Logik	<code>printf()</code> -Funktionen geben PASS/FAIL-Meldungen aus

Best Practices

- Ausschließlich Nicht-Produktionsdaten für Tests verwenden.
- CI/CD-Integration (z. B. GitHub Actions) einrichten, um Tests bei jeder Codeänderung automatisch laufen zu lassen.
- Testergebnisse über Cluster-Treiber-Logs oder konfigurierte Log-Weiterleitung nachverfolgen.
- Temporäre Views nach dem Testen wieder löschen.
- Unit-Tests lassen sich zusammen mit Notebook-Jobs automatisiert einplanen.

Hinweis: Web-Terminals unterstützen kein Spark und eignen sich daher nicht für umfassende Tests – Notebooks sind die empfohlene Umgebung.

Quelle: <https://docs.databricks.com/aws/en/notebooks/testing>
Stand: 2026-08-06